

ADDRESS RELATIVE JUMPING TABLE:

+25	19	+18	12	+11	0B	+4	04	-3	FD	-10	F6	-17	EF
+24	18	+17	11	+10	0A	+3	03	-4	FC	-11	F5	-18	EE
+23	17	+16	10	+9	09	+2	02	-5	FB	-12	F4	-19	ED
+22	16	+15	0F	+8	08	+1	01	-6	FA	-13	F3	-20	EC
+21	15	+14	0E	+7	07	0	00	-7	F9	-14	F2	-21	EB
+20	14	+13	0D	+6	06	-1	FF	-8	F8	-15	F1	-22	EA
+19	13	+12	0C	+5	05	-2	FE	-9	F7	-16	F0	-23	E9

SPECIAL FUNCTION REGISTERS:

Table 1

Symbol	Name	Address
*ACC	Accumulator	0E0H
*B	B Register	0F0H
*PSW	Program Status Word	0D0H
SP	Stack Pointer	81H
DPTR	Data Pointer 2 Bytes	
DPL	Low Byte	82H
DPH	High Byte	83H
*P0	Port 0	80H
*P1	Port 1	90H
*P2	Port 2	0A0H
*P3	Port 3	0B0H
*IP	Interrupt Priority Control	0B8H
*IE	Interrupt Enable Control	0A8H
TMOD	Timer/Counter Mode Control	89H
*TCON	Timer/Counter Control	88H
*+ T2CON	Timer/Counter 2 Control	0C8H
TH0	Timer/Counter 0 High Byte	8CH
TL0	Timer/Counter 0 Low Byte	8AH
TH1	Timer/Counter 1 High Byte	8DH
TL1	Timer/Counter 1 Low Byte	8BH
+ TH2	Timer/Counter 2 High Byte	0CDH
+ TL2	Timer/Counter 2 Low Byte	0CCH
+ RCAP2H	T/C 2 Capture Reg. High Byte	0CBH
+ RCAP2L	T/C 2 Capture Reg. Low Byte	0CAH
*SCON	Serial Control	98H
SBUF	Serial Data Buffer	99H
PCON	Power Control	87H

* = Bit addressable

+ = 8052 only

INTERRUPT PINS:

EA	EC	ET2	ES	ET1	EX1	ET0	EX0
----	----	-----	----	-----	-----	-----	-----

Bit	Name	Description
7	EA	Interrupt Enable Bit: EA = 1 interrupt(s) can be serviced, EA = 0 interrupt servicing disabled.
6	BPE	Breakpoint Enable bit. (CV-8052)
5	ET2	Timer 2 Interrupt Enable. (8052)
4	ES	Serial Port Interrupt Enable
3	ET1	Timer 1 Overflow Interrupt Enable.
2	EX1	External Interrupt 1 Enable.
1	ET0	Timer 0 Overflow Interrupt Enable.
0	EX0	External Interrupt 0 Enable.

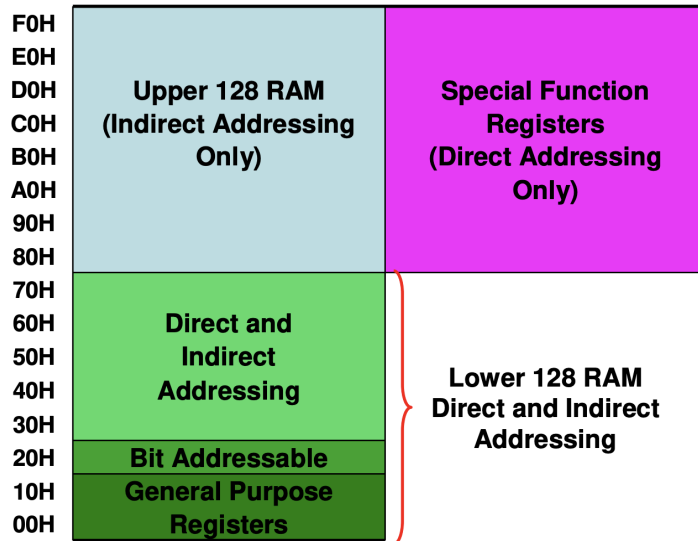
DIRECT VS. INDIRECT ADDRESSING:

DIRECT	INDIRECT
mov a, A0H	mov R0, #A0H mov a, @R0 (can only use R0 or R1)

mov a, #A0H would store the value A0H into a, whereas **mov a, A0H** stores the value at the location A0H in memory into a

#A0H is the name of the location in memory, whereas **@R0** refers to the data stored at that location

RAM (DATA MEMORY):



Use the segment directives to select bit, data, idata, xdata or code memory.

- BSEG defines bit variables
- DSEG (usually at 30H+) defines variables in data memory
 - Use DS (define store) to define memory space for byte-sized variables
- ISEG (at 80H+) defines variables in idata memory
- XSEG defines variables in xdata memory
 - Use DS to define the byte variables
- CSEG (usually at 1000H) defines constants in code memory
 - Use DB (define byte) to store constants memory

PUSH/POP FROM STACK:

Popular registers to push/pop:

- ACC, DPL, DPH (need to do these instead of DPTR), PSW, AR0 to AR7 (these are the addresses of R0 to R7)
- Pop registers from the stack in **REVERSE** order you pushed them (stack is a LIFO)

INTERRUPTS:

- To execute subroutines automatically without using "LCALL"
- Must end interrupt subroutines (interrupt service routines) with "**RETI**" instead of the regular "RET"
- When an interrupt occurs, it will go to these specific addresses for the ISRs:

Interrupt source	Address
External 0	0003H
Timer 0	000BH
External 1	0013H
Timer 1	001BH
Serial port	0023H
Timer 2	002BH

- If you enable a particular interrupt there **MUST** be an ISR (even if it's just "reti" or the program will crash!

TIMERS:

- **TIMER 0 and 1** have FOUR modes of operation (we only use 1 and 2):
 - **Mode 1:** 16-bit timer/counter
 - **Mode 2:** 8-bit auto-reload timer/counter
- TMOD timer/counter control register

Timer 1				Timer 0			
GATE	C/T*	M1	M0	GATE	C/T*	M1	M0

Bit	Name		Description
7 & 3	GATE		1: uses either INT0 or INT1 pins to enable/disable the timer/counter
6 & 2	C/T*		0: timer; 1: counter (pins T0 and T1)
All the other pins!	M1	M0	
	0	0	13-bit timer/counter
	0	1	16-bit timer/counter
	1	0	8-bit auto-reload timer/counter
	1	1	Special mode

- Timer 1 and timer 0 have the same functionality
- Non-bit addressable so must set the bits with orl:
 - orl A, #00000010B ; (sets the 7th bit to 1)
 - mov TMOD, A
- Timer Control Register:

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
-----	-----	-----	-----	-----	-----	-----	-----

Bit	Name	Description
7	TF1	Timer 1 overflow flag.
6	TR1	Timer 1 run control.
5	TF0	Timer 0 overflow flag.
4	TR0	Timer 0 run control.
3	IE1	Interrupt 1 flag.
2	IT1	Interrupt 1 type control bit.
1	IE0	Interrupt 0 flag.
0	IT0	Interrupt 0 type control bit.

CALCULATING TH0 AND TL0 AND MAX DELAY ACHIEVABLE:

- **MODE 1:**

$$\text{Rate} = \frac{\text{CLK}/12}{2^{16} - [\text{THn}, \text{TLn}]} = \frac{33.3333\text{MHz}/12}{65536 - [\text{THn}, \text{TLn}]}$$

$$[\text{THn}, \text{TLn}] = 65536 - \frac{2.77777\text{MHz}}{\text{Rate}} = 65536 - \frac{2.77777\text{MHz}}{(1/10\text{ms})} = 37758$$

Maximum delay achievable?

$$\text{Rate} = \frac{\text{CLK}/12}{2^{16} - [\text{THn}, \text{TLn}]} = \frac{2.77777\text{MHz}}{65536 - [\text{THn}, \text{TLn}]}$$

$$[\text{THn}, \text{TLn}] = 0$$

$$\text{Rate} = \frac{2.77777\text{MHz}}{65536} = 42.39\text{Hz} \rightarrow 23.59\text{ms}$$

- If in **MODE 2**, it should be 2^8 since we have 8 bits instead of 16

CONFIGURING A TIME DELAY FOR TIMER 0 (example below):

- To implement a delay using a timer we need to (x = 0 for timer 0; x = 1 for timer 1):
 - Initialize the timer using TMOD
 - Load the timer using THx and TLx
 - Clear the timer overflow flag, TFX = 0
 - Start the timer using TRx
 - Check the timer overflow flag TFX

Wait10ms:

```

; Initialize the timer
mov a, TMOD
anl a, #11110000B ; Clear bits for timer 0, keep bits for timer 1
orl a, #0000001B ; GATE=0, C/T*=0, M1=0, M0=1: 16-bit timer
mov TMOD, a
clr TR0 ; Disable timer 0
; Load the timer [TH0, TL0]=65536-(2777777/(1/10E-3))
mov TH0, #high(37758)
mov TL0, #low(37758)
clr TF0 ; Clear the timer flag
setb TR0 ; Enable timer 0

```

Wait10ms_L0:

```

jnb TF0, Wait10ms_L0 ; Wait for overflow
ret

```

I/O PORTS:

	JP1				JP2		
LCD_DATA[0]	1	2	LCD_DATA[1]	P0.0	1	2	P0.1
LCD_DATA[2]	3	4	LCD_DATA[3]	P0.2	3	4	P0.3
LCD_DATA[4]	5	6	LCD_DATA[5]	P0.4	5	6	P0.5
LCD_DATA[6]	7	8	LCD_DATA[7]	P0.6	7	8	P0.7
LCD_EN	9	10	LCD_RS	P1.0	9	10	P1.1
5V	11	12	GND	5V	11	12	GND
LCD_RW	13	14	TXD	P1.2	13	14	P1.3
LCD_ON	15	16	RXD	P1.4	15	16	P1.5
FL_DQ[0]	17	18	FL_DQ[1]	P1.6	17	18	P1.7
FL_DQ[2]	19	20	FL_DQ[3]	P2.0	19	20	P2.1
FL_DQ[4]	21	22	FL_DQ[5]	P2.2	21	22	P2.3
FL_DQ[6]	23	24	FL_DQ[7]	P2.4	23	24	P2.5
FL_RST_N	25	26	FL_WE_N	P2.6	25	26	P2.7
FL_OE_N	27	28	FL_CE_N	P3.0	27	28	P3.1
3.3V	29	30	GND	3.3V	29	30	GND
TDO	31	32	TDI	P3.2	31	32	P3.3
TCS	33	34	TCK	P3.4	33	34	P3.5
Not used	35	36	Not used	P3.6	35	36	P3.7
 T0	37	38	T1 	INT0	37	38	INT1
 T2	39	40	T2EX	Not Used	39	40	Not used

- To use the I/O as output pins configure them with the PxMOD register (not bit addressable)
 - "1" makes an output and "0" makes an input
- Ex:
 - mov P0MOD, #00000010b ; Makes P0.1 output
 - mov P1MOD, #0ffH ; All pins of P1 are now outputs
 - mov P2MOD, #00001111b ; P2.0 to P2.3 are outputs, P2.4 to P2.7 are inputs

SHORT CODE SEGMENTS:

DECREMENT THE DPTR: <pre> mov a, dpl jnz skip_dph ; jump if the acc is not zero dec dph skip_dph: dec dpl </pre>	MACRO: average MAC <pre> mov a, %0 add a, %1 add a, %2 mov b, #3 div ab end MAC </pre>
HALF-SEC DELAY SUBROUTINE: Delay: <pre> mov R2, #90 L2: mov R1, #250 L1: mov R0, #250 L0: djnz R0, L0 ; 3*30ns*250 = 22.5 us djnz R1, L1 ; 22.5us*250 = 5.625ms djnz R2, L2 ; 5.625ms*90 = 0.506s ret </pre>	INTERRUPT SETUP FOR TIMER Org 0H <pre> ljmp myprogram ; Timer 0 interrupt org 0BH ljmp myinterrupt myinterrupt: ; write some code here </pre>

	<pre> reti myprogram: mov a, TMOD anl a, #F0H orl a, #00000010B ; mode 2 mov TMOD, a mov TH0, #080H ; set interrupt rate setb TR0 ; enable timer 0 setb ET0 ; enable timer 0 interrupt setb EA ; enable global interrupts </pre>
--	---

DISPLAYING A BYTE ON THE 7-SEGMENT DISPLAYS

ORG 0

ljmp main

T_7seg:

DB 40h, 79h, 24h, 30h, 19h, 12h, 02h, 78h

DB 00h, 10h, 08h, 03h, 46h, 21h, 06h, 0Eh

Display_Byte:

mov a, r0 ; displaying least significant digit on HEX0

anl a, #00001111b ; same as anl a, #0x0F

movc a, @a+dptr

mov hex0, a

mov a, r0 ; displaying most significant digit on HEX1

anl a, #11110000b ; same as anl a, #0xF0

swap a

movc a, @a+dptr

mov hex1, a

ret

main:

mov sp, #7fH

mov dptr, #T_7seg

mov r0, #0xA3

lcall Display_Byte

COMPARING NUMBERS:

- Using single or multi-byte subtraction to compare two numbers (minuend - subtrahend):
 - If the result of the subtraction is 0, minuend = subtrahend
 - If the result is not zero, subtrahend $\neq$ subtrahend
 - If “borrow” is set, minuend < subtrahend

- If no “borrow” is set, minuend $\geq$ subtrahend
- The “borrow” is set in the carry flag
- To check, use these instructions: JZ, JNZ, JC, JNC

SUBTRACTING TWO 16-BIT NUMBERS IN BCD:

main:

```

mov sp, #7fH

; add two 16-bit numbers

; first number is [R7,R6] or in this case 1234 (decimal)
mov R7, #0x12
mov R6, #0x34

; second number is [R5,R4] or in this case 4567 (decimal)
mov R5, #0x45
mov R4, #0x67

; [R7 R6] -
; [R5 R4]
; -----
; [R3 R2] (BCD)

; Get the 9-complement of [R5 R4] (this is 99-RX)

clr c
mov a, #0x99
subb a, R5 ; computing 99-R5
mov R5, a

clr c
mov a, #0x99
subb a, R4 ; computing 99-R4
mov R4, a

setb c ; Because we need minuend + 9_complement(R5R4) + 1
mov a, R6
addc a, R4
DA A ; decimal adjust, only valid after add or addc
mov R2, a

mov a, R7
addc a, R5
DA A
mov R3, a

```

BINARY TO BCD CONVERSION (DOUBLE DABBLE ALGORITHM):

1. BIN: 01011011 BCD=000

Multiply BDC by two and add the underlined bit:
 $BCD=(000+000+0)=000$

2. BIN: 01011011 BCD=000

Multiply BDC by two and add the underlined bit:
 $BCD=(000+000+1)=001$

3. BIN: 01011011 BCD=001

Multiply BDC by two and add the underlined bit:
 $BCD=(001+001+0)=002$

4. BIN: 01011011 BCD=002

Multiply BDC by two and add the underlined bit:
 $BCD=(002+002+1)=005$

5. BIN: 01011011 BCD=005

Multiply BDC by two and add the underlined bit:
 $BCD=(005+005+1)=011$

6. BIN: 01011011 BCD=011

Multiply BDC by two and add the underlined bit:
 $BCD=(011+011+0)=022$

7. BIN: 0101101 BCD=022

Multiply BDC by two and add the underlined bit:
 $BCD=(022+022+1)=045$

8. BIN: 01011011 BCD=045

Multiply BDC by two and add the underlined bit:
 $BCD=(045+045+1)=091$

← Final result is 9

CONVERTING HEX NUMBER TO BCD NUMBER

; Converts the 16-bit hex number in R0-R1 to BCD in R2-R3-R4

hex2bcd:

mov R2, #0 ; Set BCD result to 00000

mov R3, #0

mov R4, #0

mov R5, #16 ; Loop counter.

L0:

mov a, R1 ; Shift R0-R1 left through carry

rlc a

mov R1, a

mov a, R0

rlc a

mov R0, a

mov a, R4

addc a, R4 ; Perform bcd + bcd + carry using BCD numbers

da a

mov R4, a

mov a, R3

addc a, R3

da a

mov R3, a

mov a, R2

addc a, R2

mov R2, a

djnz R5, L0

ret

main:

; Load to [R0,R1] 0xffff

mov R0, #0xff

mov R1, #0xff

```
; Concert to bcd
lcall hex2bcd
```

BCD TO 8-BIT BINARY CONVERSION (DOUBLE DABBLE ALGORITHM):

1. BCD=137 BIN: 00000000
Multiply BIN by ten and add the underlined digit:
BIN=((00000000*1010)+0001)=00000001
2. BCD=137 BIN: 00000001
Multiply BIN by ten and add the underlined digit:
BIN=((00000001*1010)+0011)=00001101
3. BCD=137 BIN: 00001101
Multiply BIN by ten and add the underlined digit:
BIN=((00001101*1010)+0111)=**10001001**

Convert a three digit BCD number (0 to 255) stored in R6-R7 to binary and save the result in R5.

<pre>mov a, R6 mov b, #10 mul ab mov R5, a mov a, R7 swap a anl a, #0fH add a, R5 mov b, #10 mul ab mov R5, a</pre>	<pre>mov a, R7 anl a, #0fH add a, R5 mov R5, a</pre>
---	--

BCD TO N-BIT BINARY CONVERSION:

Convert 205 to binary:

```
205 ÷ 2 = 102 R = 1
102 ÷ 2 = 51 R = 0
51 ÷ 2 = 25 R = 1
25 ÷ 2 = 12 R = 1
12 ÷ 2 = 6 R = 0
6 ÷ 2 = 3 R = 0
3 ÷ 2 = 1 R = 1
1 ÷ 2 = 0 R = 1
```

To convert BCD to binary we need to divide a BCD number by two! This algorithm works in reverse from the double dabble algorithm!!!

205 = 11001101B = 0CDH

Divide BCD number 3925 by 2:

0011 1001 0010 0101 BCD number to divide by 2

0001 1100 1001 0010 Right shift

0000 0011 0011 0000 Correction

0001 1001 0110 0010 Subtract

1 9 6 2

3925 / 2 = 1962, remainder=1

- Shift the number to the right by 1
- If the most significant bit of the BCD number is 1, subtract that digit by 3 (correction)
- The least significant bit of the least significant BCD digit, is the remainder (1, in this case, cause 0101 ends with a 1)
- Keep doing this to convert the BCD to binary

```
; bcd2hex: Converts the BCD number in R2-R3-R4  
; to binary in R0-R1
```

```
rrc_and_correct:
```

```
rrc a
```

```
mov r6, psw ; Save carry (it is changed by the add)
```

```
jnb acc.7, nocor1
```

```
add a, #(100H-30H) ; subtract 3 from packed BCD MSD
```

```
nocor1:
```

```
jnb acc.3, nocor2
```

```
add a, #(100H-03H) ; subtract 3 from packed BCD LSD
```

```
nocor2:
```

```
mov psw, r6 ; Restore carry
```

```
ret
```

```
bcd2hex:
```

```
mov R5, #16 ;Loop counter.
```

```
L0:
```

```
; Divide BCD by two
```

```
clr c
```

```
mov a, R2
```

```
lcall rrc_and_correct
```

```
mov R2, a
```

```
mov a, R3
```

```
lcall rrc_and_correct
```

```
mov R3, a
```

```
mov a, R4
```

```
lcall rrc_and_correct
```

```
mov R4, a
```

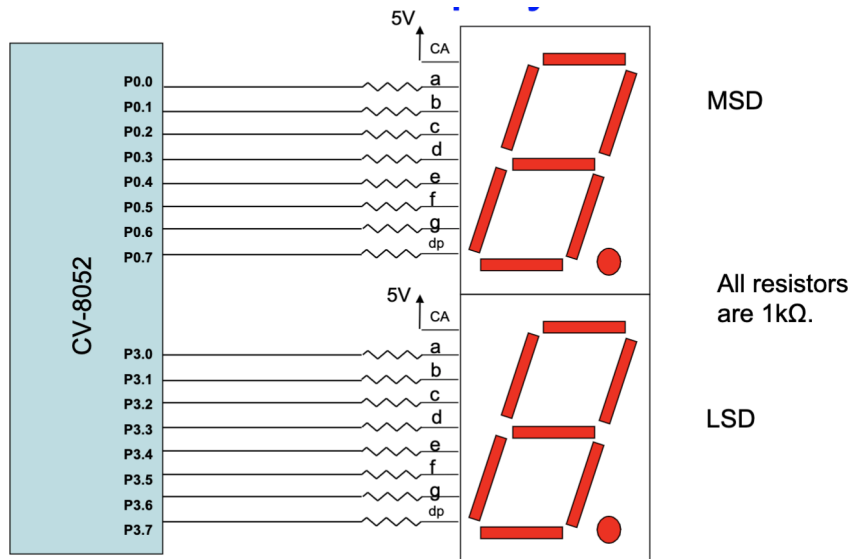
```

;Shift R0-R1 right through carry
mov a, R0
rrc a
mov R0, a
mov a, R1
rrc a
mov R1, a

djnz R5, L0
ret

```

EXAMPLE: WRITING TO 7-SEGMENT LED DISPLAYS USING I/O PORTS



\$MODDE0CV

org 0

ljmp mycode

7SegTable:

DB 0C0H, 0F9H, 0A4H, 0B0H, 099H

DB 092H, 082H, 0F8H, 080H, 090H

Wait1Sec:

mov R2, #180

L3: mov R1, #250

L2: mov R0, #250

L1: djnz R0, L1 ; 3 machine cycles-> 3*30ns*250=22.5us

djnz R1, L2 ; 22.5us*250=5.625ms

djnz R2, L3 ; 5.625ms*180=1s (approximately)

ret

mycode:

mov SP, #7FH ; initialize stack pointer

mov P0MOD, #0FFH ; configure all P0 pins as outputs

mov P3MOD, #0FFH ; configure all P3 pins as outputs

```

    mov B, #0 ; counter
    mov dptr, #7SegTable
forever:
    mov A, B ; display MSB
    swap A
    anl A, #0FH ; keep only MSB
    movc A, @A+dptr
    mov P0, A
    mov A, B ; display LSB
    anl A, #0FH
    movc A, @A+dptr
    mov P3, A
    lcall Wait1Sec ; do the one-second delay
    mov A, B ; increment counter (register B) and convert to BCD
    add A, #1
    da A
    mov B, A
    cjne A, #060H, forever ; Check if count is 60
    mov B, #0 ; if count reached 60, reinitialize to 0
    sjmp forever
END

```

SOFTWARE DEBOUNCING DUE TO BUTTON PRESSING:

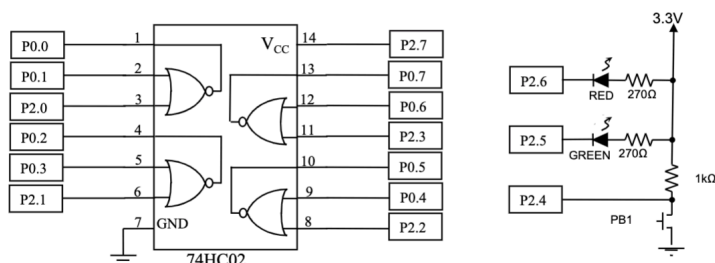
```

; After reset all ports are inputs
jb P0.0, not_pressed
lcall Wait50ms ; Wait and check again
jb P0.0, not_pressed
; Wait for the button to be released
L0: jnb P0.0, L0
sjmp pressed

```

LOGIC IC TESTING:

The circuit in the figure below can be used to verify that a 74HC02 integrated circuit (IC) operates correctly. The 74HC02 IC consists of four 2-input NOR gates. Write a subroutine for the CV-8052 processor that tests the IC after push button PB1 is pressed and then released (no de-bouncing needed). Additionally, the subroutine should: configure the input and output pins, apply power to the IC, test all possible input/output combinations and either turn the green LED on if the IC passes all the tests or the red LED on if the IC fails any test. The subroutine should set the power pin as well as all of the IC inputs to zero before returning so that the IC can be removed safely from the circuit after the tests are completed.



Pins that are outputs from the CV-8052 (inputs to the IC):

- P0.1, P2.0, P0.3, P2.1, P2.2, P0.4, P2.3, P0.6, P2.5, P2.6, P2.7

Pins that are inputs to the CV-8052 (outputs from the IC):

- P0.0, P0.2, P0.5, P0.7, P2.4

Test_NOR_IC:

```
mov P0MOD, #01011010b ; configuring input/output pins
mov P2MOD, #11101111b
jb P2.4, $ ; Wait for PB1 press (goes to next line when button is pressed, low logic)
mov P2, #11100000B
; power on, red LED off, green LED off (logic high = no current flow)
jnb P2.4, $ ; Wait for PB1 release
```

; testing 4 cases of the NOR gate (in table for efficient space usage!)

<pre>; 0 nor 0 = 1 clr P0.1 ; configure both inputs to NOR gate to 0 clr P2.0 jnb P0.0, error_NOR ; if NOR output is not 1, jump to error_NOR clr P0.3 clr P2.1 jnb P0.2, error_NOR clr P0.4 clr P2.2 jnb P0.5, error_NOR clr P2.3 clr P0.6 jnb P0.7, error_NOR</pre>	<pre>; 0 nor 1 = 0 clr P0.1 setb P2.0 jb P0.0, error_NOR clr P0.3 setb P2.1 jb P0.2, error_NOR clr P0.4 setb P2.2 jb P0.5, error_NOR clr P2.3 setb P0.6 jb P0.7, error_NOR</pre>
<pre>; 1 nor 0 = 0 setb P0.1 clr P2.0 jb P0.0, error_NOR setb P0.3 clr P2.1 jb P0.2, error_NOR setb P0.4 clr P2.2 jb P0.5, error_NOR setb P2.3 clr P0.6 jb P0.7, error_NOR</pre>	<pre>; 1 nor 1 = 0 setb P0.1 setb P2.0 jb P0.0, error_NOR setb P0.3 setb P2.1 jb P0.2, error_NOR setb P0.4 setb P2.2 jb P0.5, error_NOR setb P2.3 setb P0.6 jb P0.7, error_NOR</pre>

```
clr P2.5 ; if this line is reached: success, all tests pass
sjmp done
```

error_NOR:

```
clr P2.6 ; At least one test fails
```

done:

```
    anl P0, #10100101B ; Set all the IC inputs to zero
    anl P2, #11110000B ; Set all the IC inputs to zero
    clr P2.7 ; Turn power off
    ret
```

GENERATING A 1kHz SQUARE WAVE WITH TIMER 0 ISR:

\$MODDE0CV

org 0

ljmp mycode

org 0bh

ljmp Timer0_ISR

Init_Timer_0:

```
    anl TMOD, #0F0H ; Mask the bits of timer 0
    orl TMOD, #01H ; Mode 1
    clr TF0 ; Overflow flag=0
    mov TH0, #0xff ; Interrupt immediately
    mov TL0, #0xff ; Interrupt immediately
    setb ET0 ; Enable overflow interrupt
    setb EA ; Enable global interrupts
    setb TR0 ; Start timer/counter 0
    ret
```

CLK EQU 33333333

FREQU_OUT EQU 1000

TIMER_0_RELOAD EQU (65536-(CLK/(12*(FREQU_OUT*2))))

Timer0_ISR:

```
    clr TF0 ; Not needed for timer 0
    mov TL0, #low(TIMER_0_RELOAD)
    mov TH0, #high(TIMER_0_RELOAD)
    cpl P0.0
    reti
```

mycode:

```
    mov SP, #7FH
    clr a
    mov LEDRA, a
    mov LEDRB, a
    mov P0MOD, #00000001B ; Make P0.0 output
    lcall Init_Timer_0
```

forever:

sjmp forever

END

PAST EXAM PROBLEMS:

Q4: Write an assembly subroutine for the 8051 microcontroller to perform the operation

Z=X-Y, where Z, X, and Y are packed BCD numbers defined as:

DSEG at 040H

X: DS 3 ; six BCD digits for input X

Y: DS 3 ; six BCD digits for input Y

XSEG at 4000H

Z: DS 3 ; six BCD digits for result Z

Assume the least significant BCD digits are stored at the lowest memory location for all the variables. Tip: $X-Y = X + \text{nine_complement}(Y) + 1$.

Program:

mov DPTR, #Z

mov R0, #X

mov R1, #Y

mov R2, #2 ; Bytes remaining to read -1

setb c ; set the carry bit to 1 for the $X + \text{comp}(Y) + 1 \leftarrow \text{carry bit}$

subbyte:

mov a, @R1 ; Get a Y byte

push psw ; save the carry bit, prob some better way

cpl A

**add a, #9AH ; 9's complement - flip bits, add 10 to lower nibble,
; produces a carry to high nibble + 9 = + 10 to high**

pop psw ; get the earlier carry back

addc a, @R0 ; Perform $X_i + \text{Comp}(Y_i)$ with carry

DA A ; Use Da to sort it all out back to BCD

movx @dptr, a ; Save the result to Z

inc R0 ; Increment X pointer

inc R1 ; Increment Y pointer

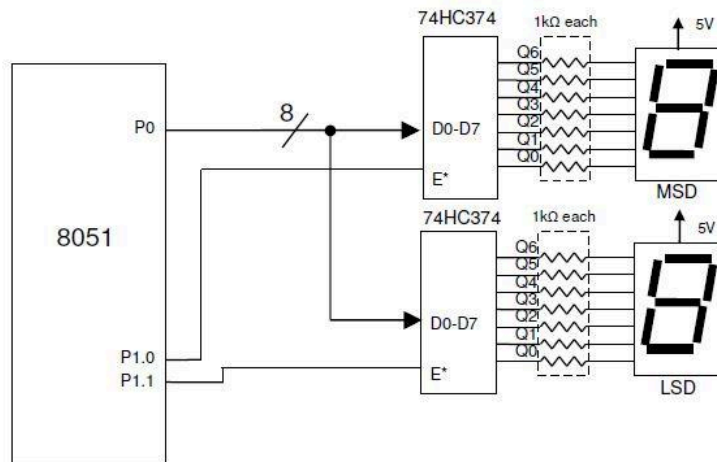
inc DPTR ; Increment Z pointer

djnz R2, subbyte ; Repeat 2x more times (3x total for 3 bytes)

Ret

Q5: A little league baseball scoreboard includes the circuit shown in the figure below to count the number of pitches the current pitcher has thrown. Write an 8051 assembly

subroutine to increment the number of pitches stored in register B (in BCD) and display the result using the common-anode 7-segment displays wired as shown in the circuit.



```
Dec7Seg:
    DB 40H, 79H, 24H, 30H, 19H, 12H, 02H, 78H, 00H, 10H
```

```
inc_disp:
    mov a, b ;set a to pitches
    clr c
    add a, #1 ; use add not inc to use DA after
    da a ; adjust back to BCD and detect rollover to 100
    jnc, skip_rollover
    mov a, #0 ; roll over 99 to 0

skip_rollover:
    mov b, a ;update b
    mov dptr, #T_7SEG
    anl a, #F0H ; get MSB
    swap a
    movc a, @a + dptr ; fetch corresponding bcd display to MSB
    mov P0, A ; prepare latch
    cpl P1.0
    cpl P1.0 ; Latch

    mov a, b ;get pitches
    anl a, #0FH ; get LSB
    movc a, @a + dptr ; fetch corresponding bcd display to LSB
    mov P0, A ; prepare latch
    cpl P1.1
    cpl P1.1 ; Latch
```

Q6: The two look-up tables below can be used to quickly convert a register to its hexadecimal ASCII representation. Write an assembly subroutine for the 8051 microcontroller to convert the value passed in register B to its hexadecimal ASCII representation and store the two ASCII values into registers R6 and R7 where R6 is the least significant digit. If variable UPPER is set to '1' use upper case letters. Otherwise use lower case letters.

```

BSEG
UPPER: DBIT 1
CSEG
TO_HEX_UPPER: DB '0123456789ABCDEF'
TO_HEX_LOWER: DB '0123456789abcdef'

```

```

convert:
mov dptr, #TO_HEX_UPPER
jb upper, DPSET      ; Skip setting to lower if upper = 1
mov dptr, #TO_HEX_LOWER ; Set to lower case
dpset:
mov a, b
anl a, #0FH          ;get lsb nibble
movc a, @a+dptr ;get conversion
mov R6, a             ;store in R6
-----
mov a, b
anl a, #F0H          ;get msb nibble
swap a
movc a, @a+dptr ;get conversion
mov R7, a           ;store in R7
ret

```

Q7: The 8051 assembly subroutine below configures timer/counter 0 as a 16-bit counter.

Write an Interrupt Service Routine for counter 0 that:

- a) Increments a 16-bit variable defined in the ISEG called 'ovf_count'. Assume the variable is located in memory above address 0x7F.**
- b) Preserves the value of all the used registers.**
- c) Returns properly.**

Note: the vector address for counter 0 interrupt is 000BH.

```

Init_counter_0:
    clr EA          ; Disable interrupts
    clr TR0         ; Stop counter 0
    mov TMOD, #05H ; Configure counter 0 in mode 1
    clr TF0         ; Clear overflow flag
    mov TH0, #0
    mov TL0, #0
    clr a
    mov R0, #ovf_count
    mov @R0, a      ; Clear overflow counter low
    inc R0
    mov @R0, a      ; Clear overflow counter high
    setb TR0        ; Start counter 0
    setb ET0        ; Enable counter 0 overflow interrupt
    setb EA         ; Enable global interrupts
    ret

org 0h
ljmp Init_counter_0

org 0bh
ljmp INT0_ISR

```

INT0_ISR:

```
push psw
push acc
push ar0

mov r0, #ovf_count
mov a, @R0          ; get count via indirect addressing
add a, #1           ; add with a carry
mov @r0, A          ; store back in iseg
inc r0              ; move data pointer value
mov a, @R0          ; get count via indirect addressing
addc a, #0           ; add with a carry in bit
mov @r0, A          ; store back in iseg

pop ar0
pop acc
pop psw
reti
```

Write an assembly subroutine for the 8051 microcontroller to compute the average of eight consecutive 16-bit numbers stored from memory locations 40H to 4FH. Store the average in registers DPL (LSD) and DPH (MSD). Tip: use the 'rrc' instruction to divide a multi-byte number by a power of 2.

```
average_8:
    push PSW
    push ACC
    push AR0
    push AR1
    push B
    mov DPTR, #0
    mov B, #0
    mov R0, #0x40
    mov R1, #8
addition_loop:
    mov A, DPL
    add A, @R0
    mov DPL, A ; DPL += input_low
    inc R0
    mov A, DPH
    addc A, @R0
    mov DPH, A ; DPH += carry + input_high
    inc R0
    jnc carry_not_present ; jump if no carry
    inc B ; if there is an overflow, this will add the carry into B
    clr c
carry_not_present:
```

```

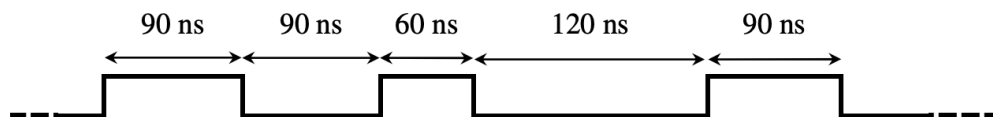
        djnz R1, addition_loop

mov R1, #3
div_8_loop:
    clr C
    mov A, B
    rrc A
    mov B, A
    mov A, DPH
    rrc A
    mov DPH, A
    mov A, DPL
    rrc A
    mov DPL, A
    djnz R1, div_8_loop

pop B
pop AR1
pop AR0
pop ACC
pop PSW
ret

```

Write a subroutine to generate the signal shown below at pin P0.0 of a CV-8052 processor running at 33.333333 MHz. The CV-8052 takes one clock period per machine cycle. Assume the pin is configured as an output, and that the signal is already set to logic zero before the subroutine is called.



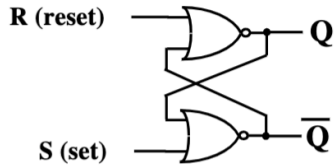
```

Subroutine:
    setb P0.0 ; 2 cycles
    NOP      ; 1 cycle
    cpl P0.0 ; 2 cycle , 1+2 = 3 On
    NOP
    cpl P0.0 ; 3 off
    cpl P0.0 ; 2 on
    NOP
    NOP
    cpl P0.0 ; 4 off
    NOP
    cpl P0.0 ; 3 on
    ret

```

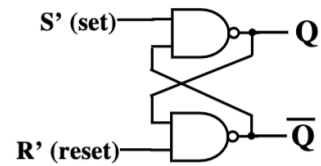
LATCHES:

SR LATCH W/ NOR GATES



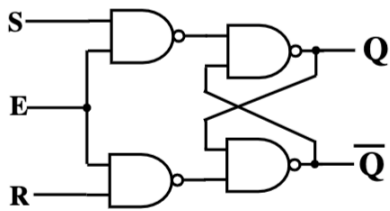
Time	R	S	Q	$\bar{Q}$	Comment
↓	0	0	?	?	Stored state unknown
	0	1	1	0	"Set" Q to 1
	0	0	1	0	Now Q "remembers" 1
	1	0	0	1	"Reset" Q to 0
	0	0	0	1	Now Q "remembers" 0
	1	1	0	0	Both go low (forbidden)
	0	0	?	?	Random!

SR LATCH W/ NAND GATES



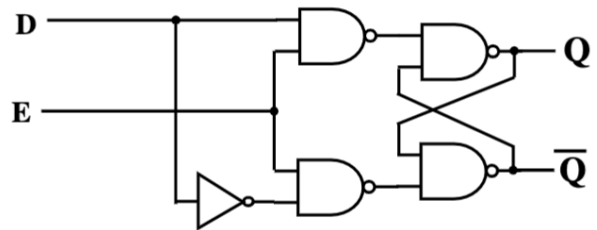
Time	R	S	Q	$\bar{Q}$	Comment
↓	1	1	?	?	Stored state unknown
	1	0	1	0	"Set" Q to 1
	1	1	1	0	Now Q "remembers" 1
	0	1	0	1	"Reset" Q to 0
	1	1	0	1	Now Q "remembers" 0
	0	0	1	1	Both go high
	1	1	?	?	Random!

GATED SR LATCH



E	S	R	Q	Q'
1	1	0	1	0
1	0	1	0	1
1	0	0	Q(t-1)	Q'(t-1)
1	1	1	?	?
0	x	x	Q(t-1)	Q'(t-1)

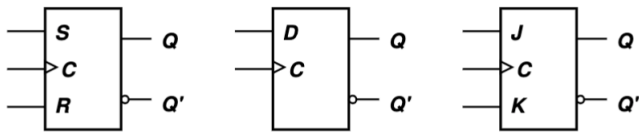
GATED D-LATCH



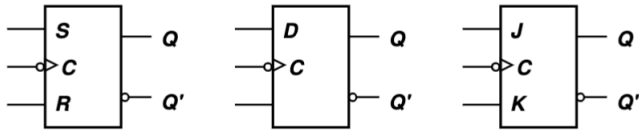
E	S	R	Q	Q'
1	1	0	1	0
1	0	1	0	1
0	x	x	Q(t-1)	Q'(t-1)

FLIP FLOPS:

- Symbols for the SR, D and JK edge-triggered flip-flops. Note the ">" symbol at the clock input.

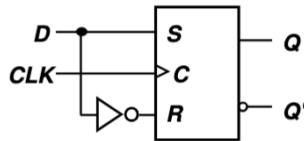


Positive edge-triggered flip-flops



Negative edge-triggered flip-flops

D FLIP-FLOP

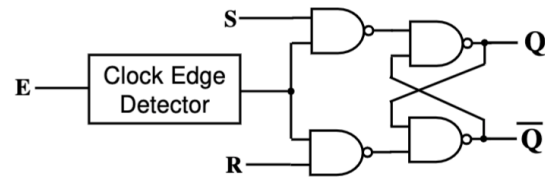


A positive edge-triggered D flip-flop formed with an S-R flip-flop.

D	CLK	Q(T+1)	Comment
1	↑	1	Set
0	↑	0	Reset

↑ = clock transition LOW to HIGH

SR FLIP-FLOP

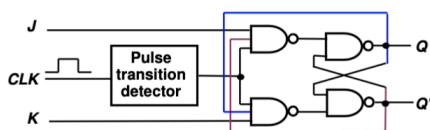


CLK	S	R	Q	Q'	Comment
?	0	0	Q(t-1)	Q'(t-1)	No Change
↑	0	1	0	1	Reset
↑	1	0	1	0	Set
↑	1	1	?	?	Invalid

? = irrelevant ("don't care")

↑ = clock transition LOW to HIGH

JK FLIP-FLOP

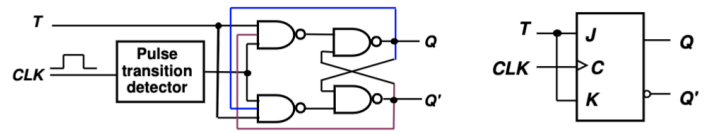


J	K	CLK	Q(t+1)	Comments
0	0	↑	Q(t)	No change
0	1	↑	0	Reset
1	0	↑	1	Set
1	1	↑	Q(t)'	Toggle

Q	J	K	Q(t+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$Q(t+1) = J \cdot Q' + K' \cdot Q$$

T FLIP-FLOP



T	CLK	Q(t+1)	Comments
0	↑	Q(t)	No change
1	↑	Q(t)'	Toggle

Q	T	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0